

TP5 – Architecture Distribuée et Sécurité

Système de Gestion de Documents Distribué (SGDD)

Aymane Es-salehy

21 avril 2026

1 TP5.1 – Architecture distribuée

1.1 Services identifiés

Les services principaux identifiés sont :

- Auth Service : gestion des utilisateurs et authentification (JWT)
- Stockage Service : gestion des documents et métadonnées
- Recherche Service : indexation et recherche
- API Gateway : point d'entrée unique
- Notification Service (optionnel)

Chaque service possède une responsabilité bien définie et fonctionne de manière indépendante.

1.2 Protocoles de communication

- Client → Gateway : HTTP REST (HTTPS)
- Gateway → Services : HTTP REST
- Stockage → Recherche : communication asynchrone (file de messages)
- Stockage → Notification : Pub/Sub

1.3 Bases de données

Chaque service possède sa propre base de données :

- Auth : PostgreSQL
- Stockage : MongoDB + MinIO
- Recherche : Elasticsearch
- Cache : Redis

1.4 Parallélisme

Certaines opérations sont exécutées en parallèle :

- Indexation après upload
- Envoi de notifications

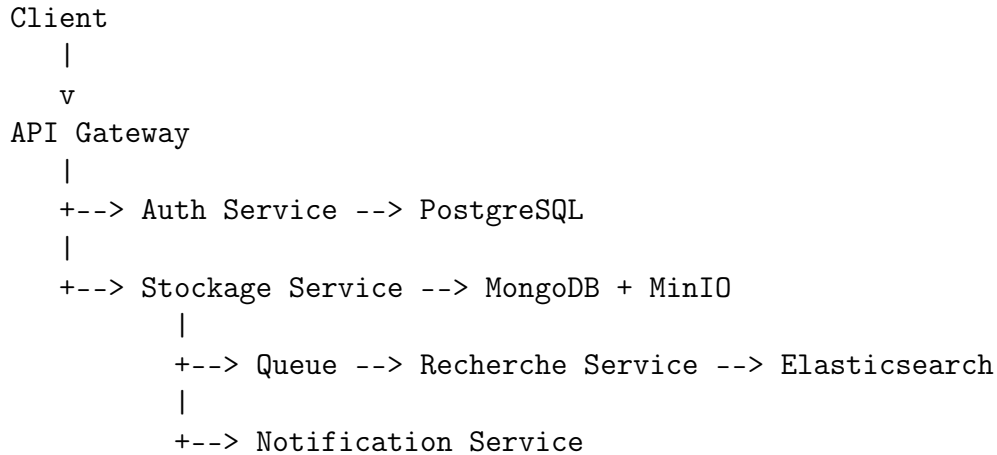
1.5 Chemins critiques

Upload : Client → Gateway → Auth → Stockage

Download : Client → Gateway → Auth → Stockage

Recherche : Client → Gateway → Auth → Recherche

1.6 Architecture logique



1.7 Architecture physique

Déploiement proposé :

- Machine 1 : API Gateway
 - Machine 2 : Services (Auth, Stockage, Recherche, Notification)
 - Machine 3 : Bases de données (PostgreSQL, MongoDB, Redis, Elasticsearch)
- Chaque service peut être déployé dans un conteneur Docker.

1.8 Conclusion

Cette architecture microservices permet une meilleure scalabilité, une isolation des composants et une tolérance aux pannes accrue. L'utilisation de communications asynchrones améliore les performances globales.

2 TP5.2 – Analyse des défis distribués

Composant	Défi	Risque	Solution
Auth Service	Disponibilité	Système bloqué	Réplication + cache JWT
Auth Service	Latence	Login lent	Cache Redis
Stockage Service	Cohérence	Document non indexé	File de messages
Stockage Service	Disponibilité	Perte accès fichiers	Réplication + backup
Recherche Service	Latence	Recherche lente	Cache + optimisation index
Recherche Service	Cohérence	Résultats obsolètes	Indexation asynchrone
API Gateway	Disponibilité	Service inaccessible	Load balancing
API Gateway	Sécurité	Attaques DDoS	Rate limiting + WAF
Notification Service	Fiabilité	Messages perdus	File persistante
Base de données	Cohérence	Données incohérentes	Transactions
Cache Redis	Cohérence	Données périmées	Invalidation cache

3 TP5.3 – Surfaces d'attaque

Surface	Menace	Contrôle	Priorité
API Gateway	DDoS, injection	WAF + rate limiting	Critique
API Gateway	Brute force	Blocage après N essais	Critique
Inter-services	MITM	mTLS	Critique
Inter-services	Usurpation	Vérification JWT	Critique
Base de données	Injection SQL	Requêtes paramétrées	Critique
Base de données	Accès externe	Réseau privé	Critique
Administration	Accès non autorisé	MFA + VPN	Élevée
Logs	Fuite données	Masquage	Élevée
Secrets	Fuite	Vault + rotation	Critique
Dépendances	Malware	pip-audit	Moyenne
Stockage fichiers	Accès illégal	URL signées	Critique
Cache Redis	Accès non sécurisé	Auth + réseau privé	Élevée

4 Conclusion générale

La conception d'un système distribué nécessite des compromis entre performance, cohérence et disponibilité. L'utilisation de patterns comme le cache, la réplication et les files de messages permet d'améliorer ces aspects tout en introduisant de la complexité. Une attention particulière doit être portée à la sécurité pour protéger les données et les communications.